Київський національний університет імені Тараса Шевченка

Кафедра мережевих і Інтернет технологій

Лабораторна робота №5-6

з дисципліни

«Технології створення програмних продуктів»

ШАБЛОН ADAPTER ТА ШАБЛОН PROXY

Група: МІТ-м11

Студент: Переверзев Олексій

Київ 2026

Завдання 1: Шаблон Adapter

Розробити прототип програми для відправки повідомлень різним способами. Необхідно додати можливість надсилати повідомлення через SMS і Telegram за допомогою сторонніх бібліотек SMSsender і TelegramSender без зміни коду базової бібліотеки MessageSender та клієнтської програми. Для цього потрібно написати два класи-адаптери.

```
using System;

namespace AdapterPatternTask
{
    public class MessageSender
    {
        public virtual void Send(string message, string recipient)
        {
            Console.WriteLine($"[Електронна пошта] Відправлено на {recipient}:
{message}");
        }
    }

    public class SMSsender
    {
        public void SendSMS(string message, string phoneNumber)
        {
            Console.WriteLine($"[SMS Сповіщення] Відправлено на номер {phoneNumber}:
{message}");
        }
    }

    public class TelegramSender
    {
        public void SendTelegram(string message, string username)
        {
            Console.WriteLine($"[Telegram Сповіщення] Відправлено користувачу
{username}: {message}");
        }
    }

    public class SmsAdapter : MessageSender
    {
        private SMSsender _smsSender = new SMSsender();

        public override void Send(string message, string recipient)
        {
            _smsSender.SendSMS(message, recipient);
        }
    }

    public class TelegramAdapter : MessageSender
    {
        private TelegramSender _telegramSender = new TelegramSender();

        public override void Send(string message, string recipient)
        {
            _telegramSender.SendTelegram(message, recipient);
        }
    }

    class Program
    {
        static void Main(string[] args)
```

```
        {
            Console.WriteLine("=== Робота системи сповіщень (Шаблон Adapter)
===\n");

            MessageSender emailSender = new MessageSender();
            emailSender.Send("Заявка на підключення прийнята.",
"client@domain.com");

            MessageSender smsSender = new SmsAdapter();
            smsSender.Send("Аварія на комутаторі агрегації усунена.",
"+380501234567");

            MessageSender telegramSender = new TelegramAdapter();
            telegramSender.Send("Оновлення прошивки MikroTik завершено успішно.",
"@client_username");

            Console.ReadKey();
        }
    }
}
```

Результат виконання програми:

```
=== Робота системи спов?щень (Шаблон Adapter) ===

[Електронна пошта] В?дправлено на client@domain.com: Заявка на підключення прийнята.
[SMS Спов?щення] В?дправлено на номер +380501234567: Аварія на комутаторі агрегації усунена.
[Telegram Спов?щення] В?дправлено користувачу @client_username: Оновлення прошивки MikroTik завершено успішно.
```

У завданні реалізовано шаблон Adapter, який перетворює інтерфейс одного класу в інший, очікуваний клієнтом. Було створено два класи-адаптери (SmsAdapter та TelegramAdapter), які наслідують цільовий інтерфейс MessageSender, але всередині містять екземпляри сторонніх класів і делегують їм виконання роботи. Це уможливило спільну роботу класів з несумісними інтерфейсами без втручання в існуючий клієнтський код.

Завдання 2: Шаблон Proxy

Створіть емулятор проксі-сервера для кешування веб-сторінок щоб покращити швидкість завантаження. Коли користувач запитує деяку веб-сторінку, проксі-сервер спочатку перевіряє, чи є ця веб-сторінка в кеші, і якщо так, то надсилає її користувачеві, інакше отримує її від веб-сервера.

```
using System;
using System.Collections.Generic;
using System.Threading;

namespace ProxyPatternTask
{
    public interface IWebServer //Subject
    {
        string GetPage(string url);
    }

    public class RealWebServer : IWebServer // RealSubject
    {
        public string GetPage(string url)
        {
```

```
            Console.WriteLine($"[Мережа] Встановлення з'єднання та завантаження
сторінки {url}...");

            Thread.Sleep(2000);

            return $"<html>Вміст веб-сторінки {url}</html>";
        }
    }

    public class ProxyWebServer : IWebServer // Proxy
    {
        private RealWebServer _realServer;

        private Dictionary<string, string> _pageCache = new Dictionary<string,
string>(); // Cache

        public string GetPage(string url)
        {
            if (_pageCache.ContainsKey(url))
            {
                Console.WriteLine($"[Кеш Проксі] Сторінку {url} знайдено в
локальному кеші. Миттєва віддача.");
                return _pageCache[url];
            }

            if (_realServer == null)
            {
                _realServer = new RealWebServer();
            }

            Console.WriteLine($"[Проксі] Сторінки {url} немає в кеші.
Перенаправлення запиту до реального сервера...");

            string pageContent = _realServer.GetPage(url);

            _pageCache[url] = pageContent;

            return pageContent;
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("=== Емулятор кешуючого проксі-сервера ===\n");

            IWebServer proxy = new ProxyWebServer();

            Console.WriteLine("--- Клієнт запитує www.knu.ua ---");
            Console.WriteLine($"Результат: {proxy.GetPage("www.knu.ua")}\n");

            Console.ReadKey();

            Console.WriteLine("--- Клієнт знову запитує www.knu.ua ---");
            Console.WriteLine($"Результат: {proxy.GetPage("www.knu.ua")}\n");

            Console.ReadKey();

            Console.WriteLine("--- Клієнт запитує www.google.com ---");
            Console.WriteLine($"Результат: {proxy.GetPage("www.google.com")}\n");

            Console.ReadKey();
        }
    }
```

}

Результат виконання програми:

```
=== Емулятор кешуючого прокс?-сервера ===

--- Кл?єнт запитує www.knu.ua ---
[Прокс?] Стор?нки www.knu.ua немає в кеш?. Перенаправлення запиту до реального сервера...
[Мережа] Встановлення з'єднання та завантаження стор?нки www.knu.ua...
Результат: <html>Вм?ст веб-стор?нки www.knu.ua</html>

--- Кл?єнт знову запитує www.knu.ua ---
[Кеш Прокс?] Стор?нку www.knu.ua знайдено в локальному кеш?. Миттєва в?ддача.
Результат: <html>Вм?ст веб-стор?нки www.knu.ua</html>

--- Кл?єнт запитує www.google.com ---
[Прокс?] Стор?нки www.google.com немає в кеш?. Перенаправлення запиту до реального сервера...
[Мережа] Встановлення з'єднання та завантаження стор?нки www.google.com...
Результат: <html>Вм?ст веб-стор?нки www.google.com</html>
```

Реалізовано шаблон Кешуючий Проксі (Caching Proxy), який забезпечує тимчасове зберігання результатів роботи (у даному випадку веб-сторінок). Клас ProxyWebServer виступає заступником для об'єкта RealWebServer. Він перехоплює виклики методу GetPage. Якщо сторінка вже є в словнику кешу, вона повертається миттєво, інакше виклик делегується реальному серверу, імітуючи тривале мережеве завантаження.

Висновок: під час виконання лабораторної роботи було досліджено застосування структурних шаблонів проектування для оптимізації архітектури додатків. Adapter довів свою корисність у ситуаціях, коли необхідно інтегрувати старий або сторонній код у нову систему без його модифікації. Він діє як "перехідник", адаптуючи виклики методів та забезпечуючи дотримання принципів відкритості/закритості (Open/Closed Principle). Proxy надав можливість гнучко контролювати доступ до ресурсоємних об'єктів. Завдяки ідентичному інтерфейсу, заступник був інтегрований абсолютно прозоро для клієнтського коду, що дозволило суттєво оптимізувати продуктивність шляхом кешування результатів. Ці навички дозволяють інтегрувати різнорідні компоненти та ефективно управляти ресурсами системи в реальних програмних проектах.